



Lab Activity

Week 13: Functions Calling Functions - Building Complete Programs

Duration: 60 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Design functions that call other functions
- Trace execution through chains of function calls
- Understand scope isolation between functions
- Build complete programs using the main function pattern
- Debug multi-function programs systematically

Getting Started

- Extract the given [week13_lab.zip](#) file to your desktop
- **KEY CONCEPT:** Functions can call other functions to break complex tasks into smaller pieces!
- Each function should do ONE thing well

1 Exercise 1: Understanding Function Chains (8 minutes)

Learn how functions call other functions!

Open [week13_ex1.py](#) and complete:

```
1  # Helper function - does one small job
2  def square(n):
3      """Returns n squared."""
4      return n * n
5
6  def double(n):
7      """Returns n doubled."""
8      return n * ___ # Fill in: 2
9
10 # Function that calls other functions
11 def square_then_double(x):
12     """Squares a number, then doubles the result."""
13     squared = square(x)
14     doubled = double(squared)
15     return doubled
16
17 # Test the chain
18 result = square_then_double(3)
```

```

19 print(f"square_then_double(3) = {result}") # Should be 18
20
21 # Trace through: 3 -> square(3)=9 -> double(9)=18
22
23 # Your turn: Create a chain that doubles then squares
24 def double_then_square(x):
25     """Doubles a number, then squares the result."""
26     doubled = _____(x) # Fill in: double
27     squared = _____(doubled) # Fill in: square
28     return squared
29
30 result2 = double_then_square(3)
31 print(f"double_then_square(3) = {result2}") # Should be 36
32
33 # Trace through: 3 -> double(3)=6 -> square(6)=36

```

 **Tip:** Order matters! square_then_double gives different results than double_then_square!

 **Checkpoint:** Can you trace through function chains? Show teacher!

2 Exercise 2: Environment Diagram Practice (10 minutes)

Trace execution and understand scope!

Open [week13_ex2.py](#) and complete:

```

1 def add(a, b):
2     result = a + b
3     return result
4
5 def multiply(a, b):
6     result = a * b
7     return result
8
9 def calculate(x, y):
10     sum_result = add(x, y)
11     product_result = multiply(x, y)
12     return sum_result + product_result
13
14 # What happens when we call calculate(3, 4)?
15 answer = calculate(3, 4)
16 print(f"calculate(3, 4) = {answer}")

```

 **Pair Programming:** Switch who types every 5 minutes!

Draw the environment diagram:

Global
answer: ---

calculate
x: ---
y: ---
sum_result: ---
product_result: ---

add
a: --- b: ---
result: ---

multiply
a: --- b: ---
result: ---

Questions:

1. What is result in the add function? ____
2. What is result in the multiply function? ____
3. Are these the same variable? ____ Why? _____
4. Final answer: ____

 **Memory Diagram:** Each function has its own local variables - same names don't mean same variables!

3 Exercise 3: Scope Isolation (10 minutes)

Discover why scope isolation is important!

Open [week13_ex3.py](#) and complete:

```

1  # Watch what happens with same variable names
2  def function_a():
3      x = 10
4      print(f"In function_a: x = {x}")
5      result = function_b()
6      print(f"Back in function_a: x = {x}")  # Still 10!
7      return x + result
8
9  def function_b():
10     x = 99  # Different x!
11     print(f"In function_b: x = {x}")
12     return x
13
14  # Run and observe
15  print("== Calling function_a ==")
16  final = function_a()
17  print(f"Final result: {final}")
18
19  # Predict before running!
20  # What will be printed? Write your predictions:
21  # Line 1: -----
22  # Line 2: -----
23  # Line 3: -----
24  # Line 4: -----
25
26  print("\n" + "=" * 40)
27
28  # Challenge: What happens here?
29  def outer():
30     message = "Hello"
31     inner()
32     return message
33
34  def inner():
35     message = "Goodbye"  # Does this change outer's message?
36     print(f"inner says: {message}")
37
38  result = outer()
39  print(f"outer returned: {result}")
40

```

```
41 # Will outer return "Hello" or "Goodbye"? -----
42 # Why? -----
```

💡 Tip: Variables inside functions are LOCAL - they can't see or change variables in other functions!

✅ Checkpoint: Can you explain scope isolation? Excellent!

4 Exercise 4: Building a Grade Calculator (12 minutes)

Create a complete program with multiple functions!

Open [week13_ex4.py](#) and complete:

```
1 # Helper function 1: Calculate average
2 def calculate_average(grades):
3     """Returns the average of a list of grades."""
4     total = sum(grades)
5     average = total / len(grades)
6     return average
7
8 # Helper function 2: Convert to letter grade
9 def get_letter_grade(score):
10    """Converts numeric score to letter grade."""
11    if score >= 90:
12        return 'A'
13    elif score >= 80:
14        return ___ # Fill in: 'B'
15    elif score >= 70:
16        return ___ # Fill in: 'C'
17    elif score >= 60:
18        return 'D'
19    else:
20        return 'F'
21
22 # Helper function 3: Get feedback message
23 def get_feedback(letter):
24    """Returns encouraging feedback based on grade."""
25    feedback = {
26        'A': "Excellent work! Keep it up!",
27        'B': "Good job! You're doing well!",
28        'C': "Not bad! A little more effort will help!",
29        'D': "You can do better! Let's work harder!",
30        'F': "Don't give up! Ask for help!"
31    }
32    return feedback.get(letter, "Keep trying!")
33
34 # Main processing function - calls all helpers
35 def process_student(name, grades):
36    """Process a student's grades and return full report."""
37    avg = _____(grades) # Fill in: calculate_average
38    letter = _____(avg) # Fill in: get_letter_grade
39    feedback = _____(letter) # Fill in: get_feedback
40
41    return {
42        "name": name,
43        "average": avg,
```

```

44         "letter": letter,
45         "feedback": feedback
46     }
47
48     # Test with sample data
49     student1 = process_student("Ali", [85, 90, 78, 92])
50     print(f"Student: {student1['name']}")
51     print(f"Average: {student1['average']:.1f}")
52     print(f"Grade: {student1['letter']}")
53     print(f"Feedback: {student1['feedback']}")
54
55     print("\n" + "-" * 40 + "\n")
56
57     # Process another student
58     student2 = process_student("Sara", [72, 68, 75, 70])
59     print(f"Student: {student2['name']}")
60     print(f"Average: {student2['average']:.1f}")
61     print(f"Grade: {student2['letter']}")
62     print(f"Feedback: {student2['feedback']}")

```

 **Pair Programming:** Switch who types every 5 minutes!

 **Checkpoint:** Grade calculator working? Awesome!

5 Exercise 5: The Main Function Pattern (10 minutes)

Learn how professionals structure programs!

Open [week13_ex5.py](#) and complete:

```

1  # A complete shopping cart program
2
3  def get_items():
4      """Returns a list of items with prices."""
5      return [
6          {"name": "Notebook", "price": 150},
7          {"name": "Pen", "price": 25},
8          {"name": "Eraser", "price": 10},
9          {"name": "Ruler", "price": 35}
10     ]
11
12 def calculate_total(items):
13     """Calculate total price of all items."""
14     total = 0
15     for item in items:
16         total += item["price"]
17     return total
18
19 def apply_discount(amount, percent):
20     """Apply discount and return new amount."""
21     discount = amount * (percent / 100)
22     return amount - _____ # Fill in: discount
23
24 def display_receipt(items, total, final):
25     """Display a nice receipt."""
26     print("\n" + "=" * 30)

```

```

27     print("          MAKTAB STORE")
28     print("=" * 30)
29     for item in items:
30         print(f"{item['name']:.<20} Rs.{item['price']}")
31     print("-" * 30)
32     print(f"{'Subtotal' :.<20} Rs.{total}")
33     print(f"{'Discount (10%)' :.<20} Rs.{total - final}")
34     print(f"{'TOTAL' :.<20} Rs.{final}")
35     print("=" * 30)
36     print("          Thank you for shopping!")
37     print("=" * 30)
38
39 def main():
40     """Main function - coordinates the entire program."""
41     # Step 1: Get items
42     items = get_items()
43
44     # Step 2: Calculate total
45     total = _____(items) # Fill in: calculate_total
46
47     # Step 3: Apply 10% discount
48     final = _____(total, 10) # Fill in: apply_discount
49
50     # Step 4: Display receipt
51     _____(items, total, final) # Fill in: display_receipt
52
53 # Run the program
54 main()

```

 **Tip:** The main() function reads like a summary - each line describes WHAT happens, helper functions handle HOW!

6 Exercise 6: Debugging Multi-Function Programs (10 minutes)

Learn to find bugs in complex programs!

Open [week13_ex6.py](#) and complete:

```

1  # This program has 3 BUGS! Find and fix them.
2
3  def get_numbers():
4      """Get a list of numbers from user."""
5      numbers = []
6      print("Enter 3 numbers:")
7      for i in range(3):
8          num = int(input(f"Number {i+1}: "))
9          numbers.append(num)
10     return numbers # BUG 1 was here: missing return
11
12 def find_max(numbers):
13     """Find the maximum number."""
14     if len(numbers) == 0:
15         return None
16     max_val = numbers[0]
17     for num in numbers:
18         if num > max_val:
19             max_val = num
20     return max_val # BUG 2: was missing return

```

```

21
22 def find_min(numbers):
23     """Find the minimum number."""
24     if len(numbers) == 0:
25         return None
26     min_val = numbers[0]
27     for num in numbers:
28         if num < min_val: # BUG 3: was using > instead of <
29             min_val = num
30     return min_val
31
32 def analyze(numbers):
33     """Analyze the numbers."""
34     maximum = find_max(numbers)
35     minimum = find_min(numbers)
36     average = sum(numbers) / len(numbers)
37
38     return {
39         "max": maximum,
40         "min": minimum,
41         "avg": average,
42         "range": maximum - minimum
43     }
44
45 def main():
46     """Main program."""
47     print("=== Number Analyzer ===\n")
48     nums = get_numbers()
49
50     print(f"\nYou entered: {nums}")
51
52     results = analyze(nums)
53
54     print(f"\nMaximum: {results['max']}")
55     print(f"Minimum: {results['min']}")
56     print(f"Average: {results['avg']:.2f}")
57     print(f"Range: {results['range']}")
58
59 # Test with known values first!
60 print("=== Testing with known values ===")
61 test_nums = [5, 2, 8]
62 print(f"Testing with: {test_nums}")
63 print(f"find_max should be 8: {find_max(test_nums)}")
64 print(f"find_min should be 2: {find_min(test_nums)}")
65
66 # Uncomment to run full program:
67 # main()

```

Debugging Strategy:

1. Test each function separately with known values
2. Add print statements to see what functions receive and return
3. Check that all functions have **return** statements
4. Verify the logic in each function

✔ **Checkpoint:** Found and fixed all bugs? Great debugging skills!

7 Lab Summary

✓ **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Created functions that call other functions
- ☐ Traced through function call chains
- ☐ Drew environment diagrams showing scope
- ☐ Understood scope isolation (same names, different variables)
- ☐ Built a complete program using the main function pattern
- ☐ Debugged multi-function programs systematically
- ☐ Tested functions individually before connecting them

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	☹️ Need Help	😬 Getting It	😊 Got It!
Functions calling functions	☐	☐	☐
Scope isolation	☐	☐	☐
The main() pattern	☐	☐	☐
Debugging strategies	☐	☐	☐

Why does each function have its own separate variables?

What's the benefit of breaking a program into many small functions?

What's ONE thing from Module 2 you're most proud of learning?

🏆 **Congratulations! You've completed Module 2!**

You now think like a programmer!